



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 25

Dr. Asad Mahmood

April 01, 2026

Lecture Outline

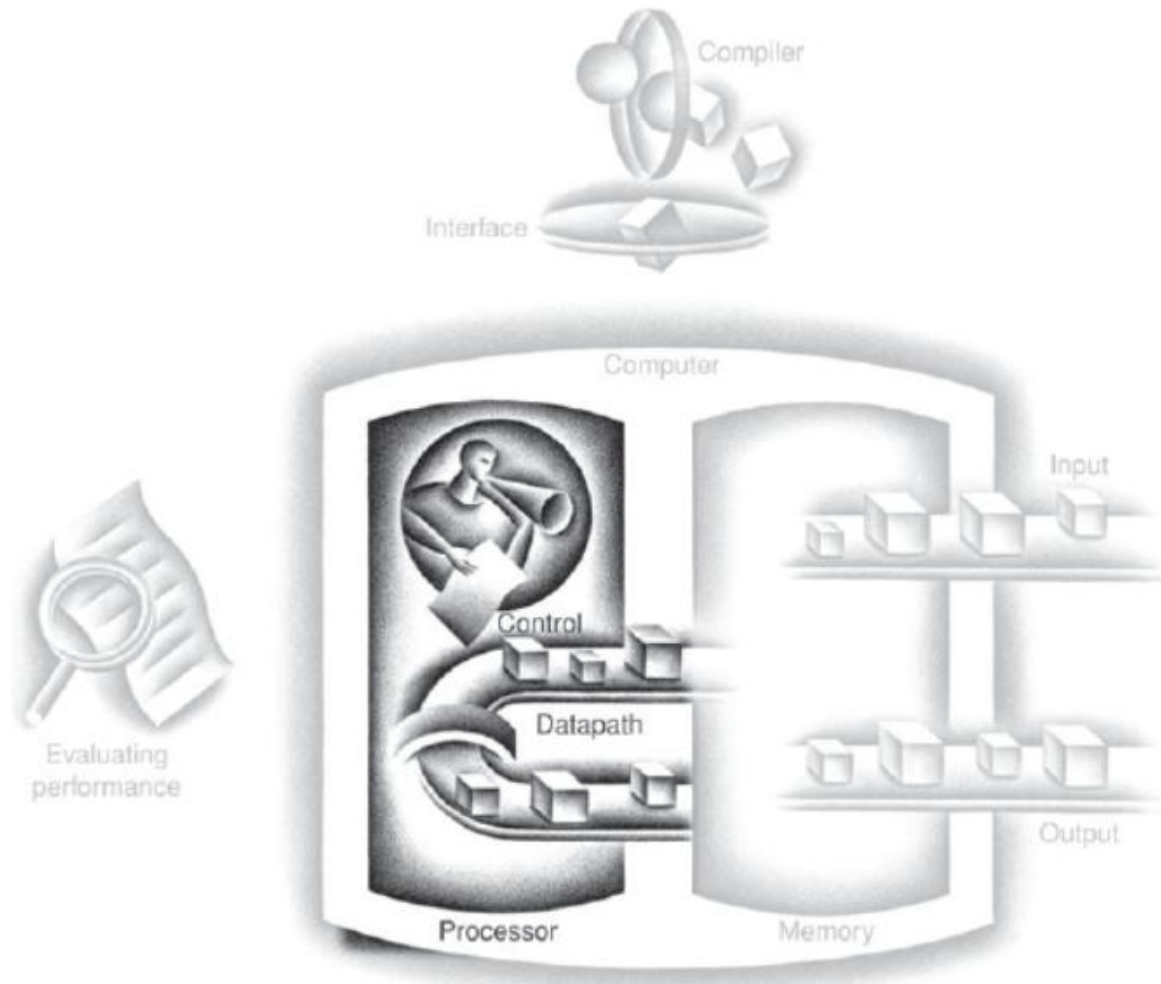
- Announcements (Assignment and Quiz 3)
- Outline of Previous Lecture:
 - **Chapter 4 – The Processor**
 - 4.1 Introduction
- Outline of Today's Lecture:
 - **Chapter 4 – The Processor**
 - 4.2 Logic Design Conventions
 - 4.3 Building a Datapath

Chapter 4

The Processor

Chapter Theme

The Five Classic Components of a Computer



Chapter Outline

Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

Introduction

Dans les grandes affaires, il n'y a pas de petits détails ! *(In a major matter, no details are small)*

French Proverb

Introduction

- CPU performance factors
 - Instruction count
 - Determined by ISA and compiler
 - CPI and Cycle time
 - Determined by CPU hardware
- We will examine two RISC-V implementations
 - A simplified version
 - A more realistic pipelined version

Introduction

- We will initially consider a subset of RISC-V ISA, enough to show most aspects of how instructions are implemented in a processor
 - Memory reference: lw, sw
 - Arithmetic/logical: add, sub, and, or
 - Control transfer: beq
- Most importantly, we will see
 - Impact of ISA on implementation
 - Impact of implementation on CPI and clock-rate
- Same concepts used to build different types (general-purpose, embedded, server etc.) of real-world processors

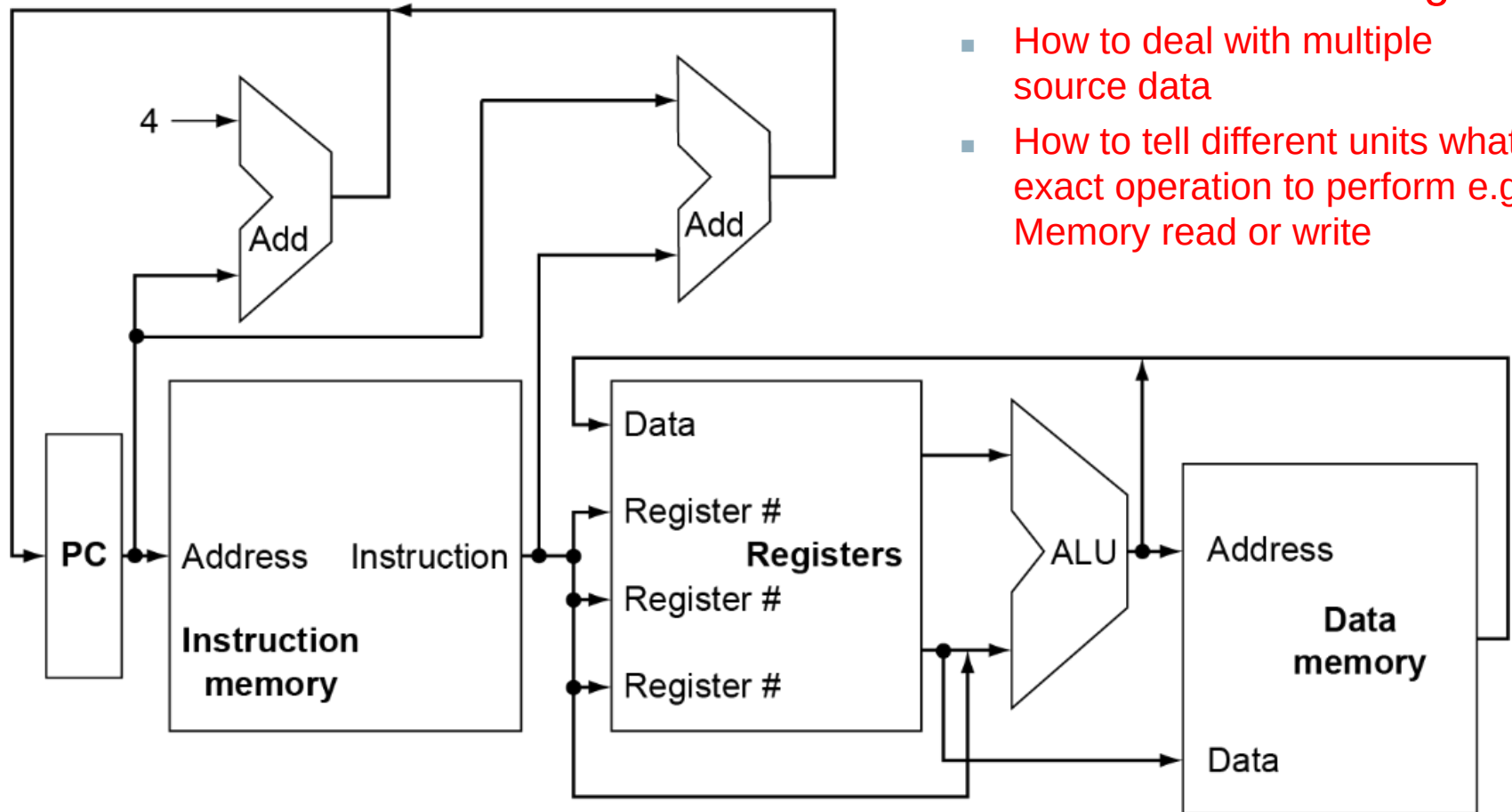
Introduction

- Let us take a brief overview of how instructions gets implemented before we delve into details in sections ahead
- Irrespective of instruction type (arithmetic, memory, branch etc.) the first two steps are common:
 1. Step 1: Fetch instruction from instruction memory using PC
 2. Step 2: Read source register/(s) mentioned in the instruction

Introduction

- Subsequent steps may differ based on instruction type, but similar for many RISC-V instructions based on principle *Simplicity favors regularity*
- For example, next step generally involves using ALU for all RISC-V instruction types:
 - Role of ALU in Arithmetic vs. Memory vs. Branch instructions
- Further next step may be different for different instruction types
 - Arithmetic vs. Memory vs. Branch instructions

CPU Overview

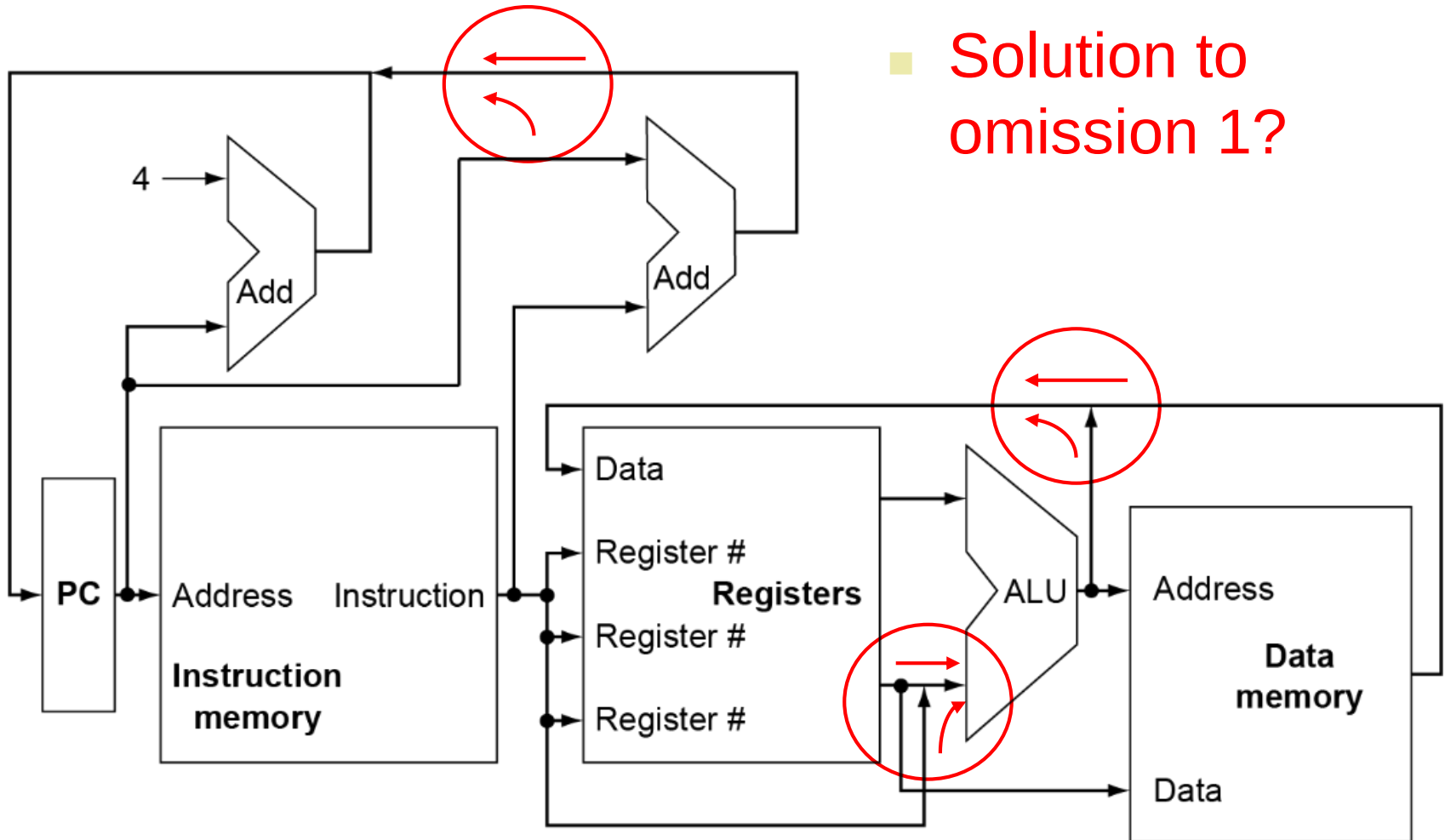


■ Two omissions in this figure:

- How to deal with multiple source data
- How to tell different units what exact operation to perform e.g. Memory read or write

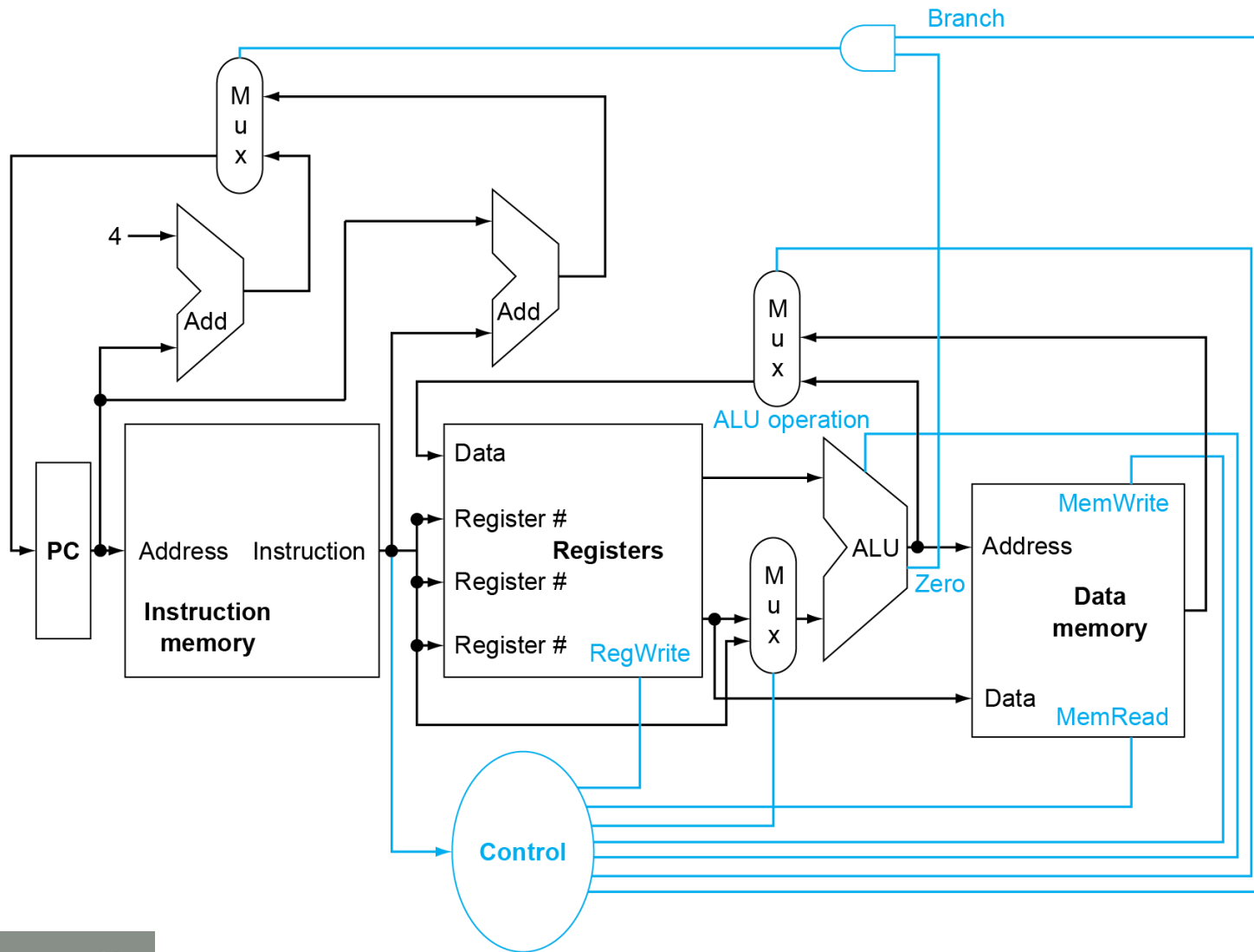
Multiplexers

- Solution to omission 1?



Control

- Solution to omission 2?



Chapter Outline

Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

Logic Design Basics

- Information encoded in binary

- Low voltage = 0, High voltage = 1

- Wire vs. **Bus**

↓ 1 Bit

↳ multiple wires

- Combinational** element

→ Depends on current input only

- Example processor unit

- State (sequential)** elements

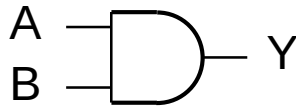
- Example processor unit

↳ Depends on current input & previous outputs

Combinational Elements

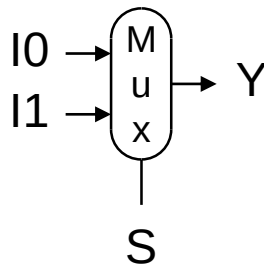
■ AND-gate

- $Y = A \& B$



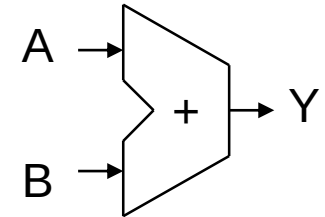
■ Multiplexer

- $Y = S ? I1 : I0$



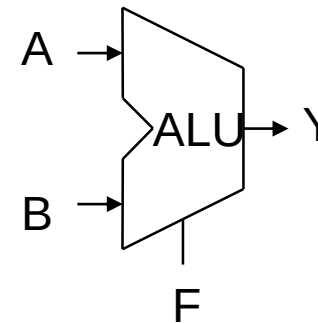
■ Adder

- $Y = A + B$



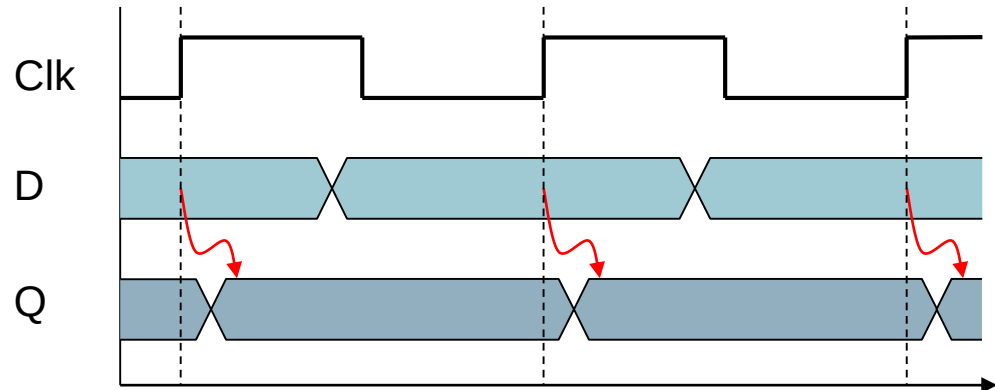
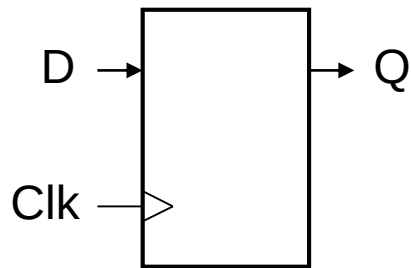
■ Arithmetic/Logic Unit

- $Y = F(A, B)$



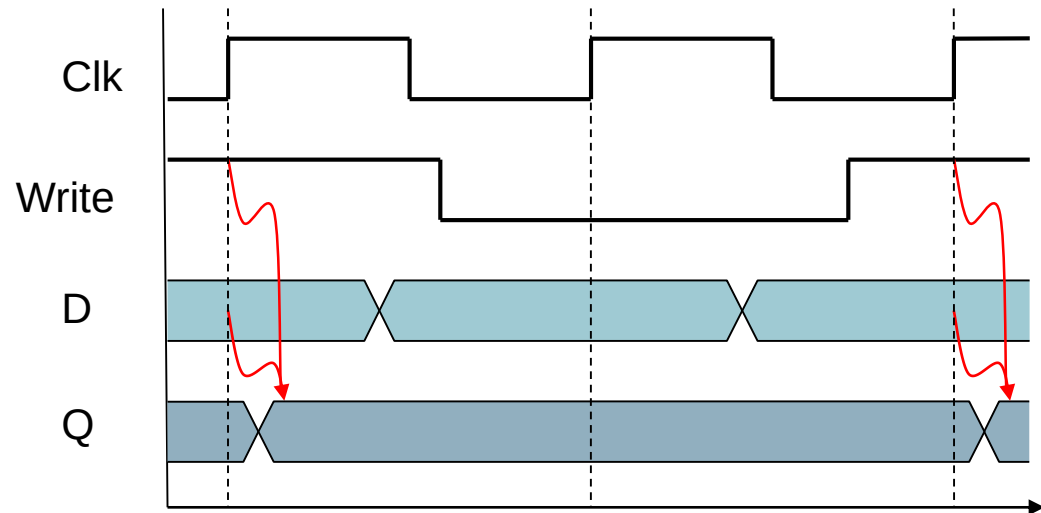
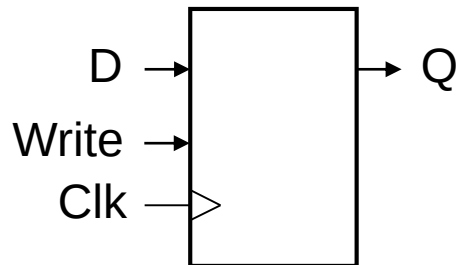
Sequential Elements

- Register: stores data in a circuit
 - Edge-triggered



Sequential Elements

- Register with **write control**
 - Only updates on clock edge when write control input is 1
 - Used when stored value is required later

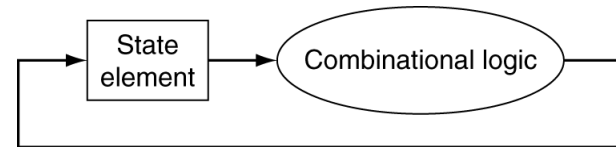
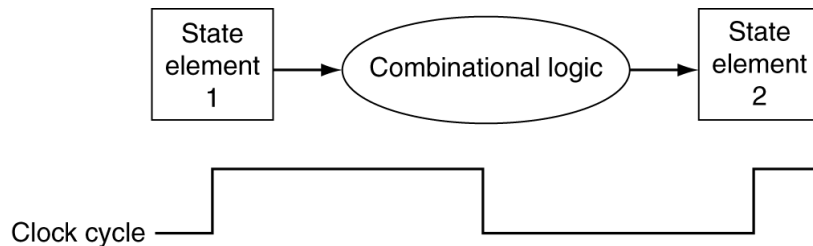


Clocking Methodology

- A clocking methodology (e.g. edge-triggered) is required/ designed to make hardware predictable
- Asserted vs. de-asserted signal

Clocking Methodology

- Combinational logic generally present between state elements
 - Combinational logic transforms data during clock cycle i.e. between clock edges
 - Longest delay determines clock period
 - Implementation impacts clock cycle!



Chapter Outline

Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

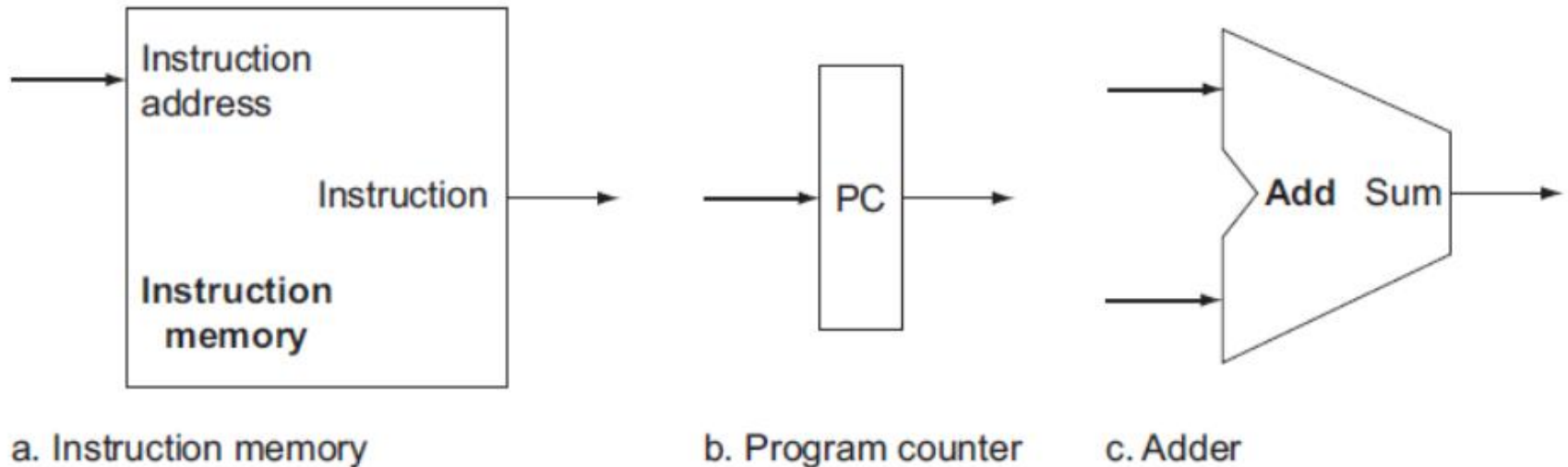
Building a Datapath

- Datapath
 - Set of elements that process data and addresses in the CPU
 - Registers, ALUs, mux's, memories, ...
- We will build a RISC-V datapath incrementally
 - Refining the overview design

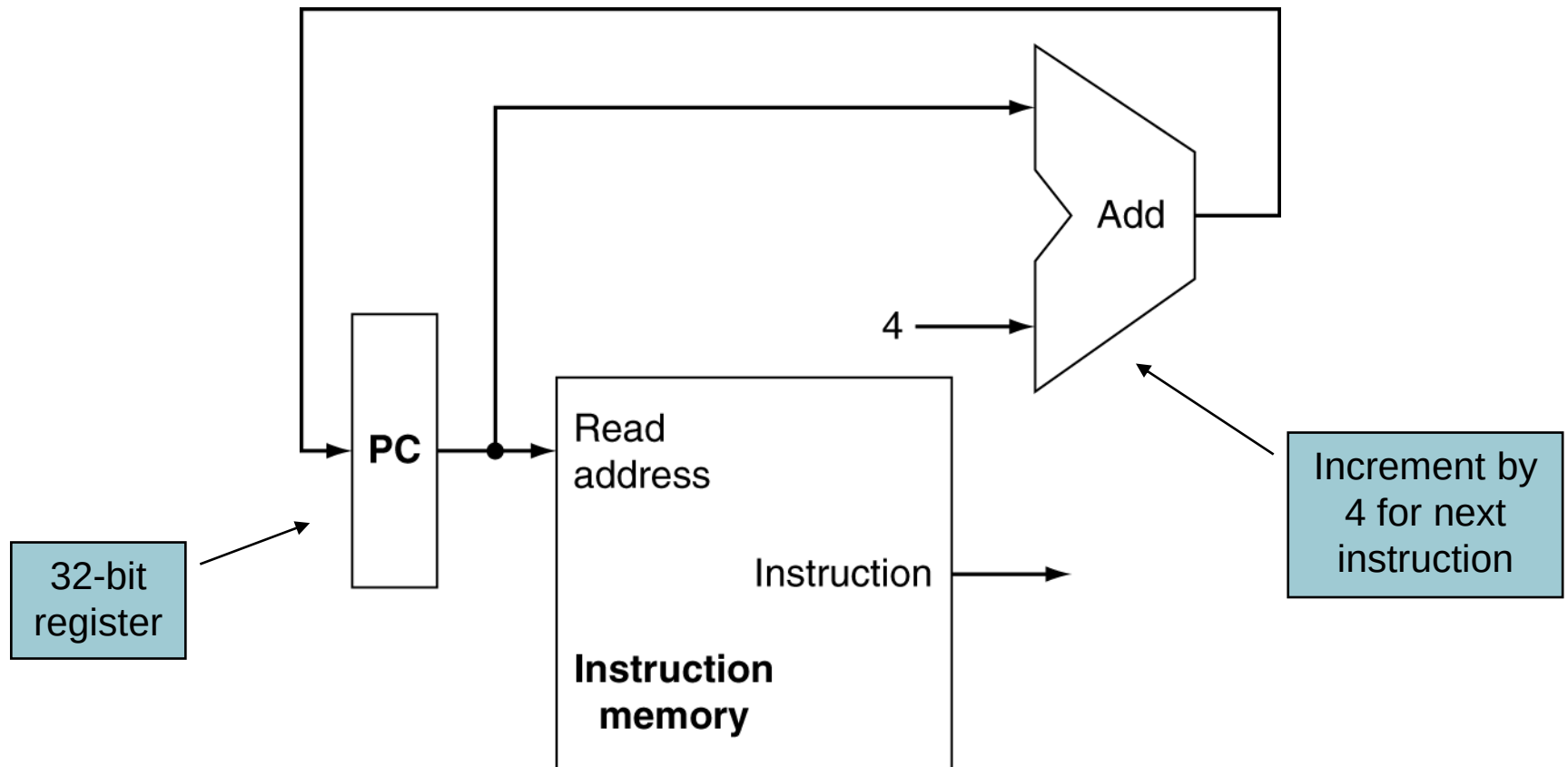
Datapath for Instruction Fetch

Datapath Elements

- PC width

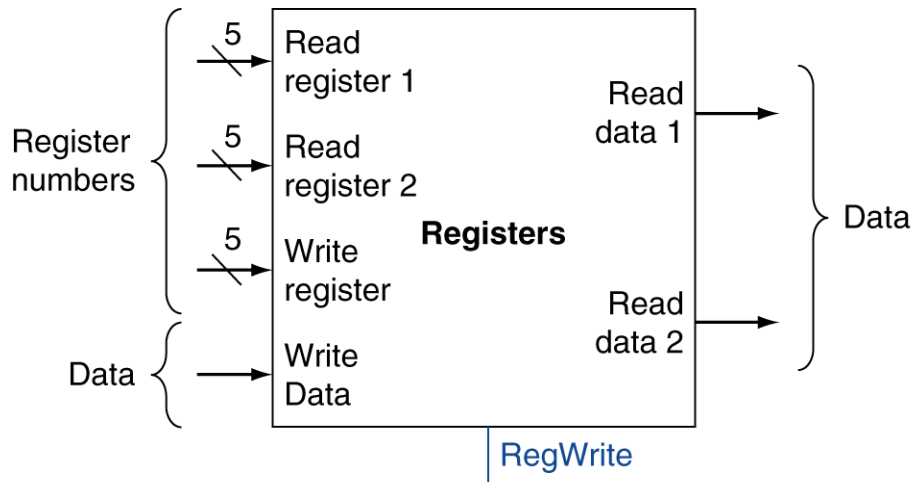


Datapath for Instruction Fetch

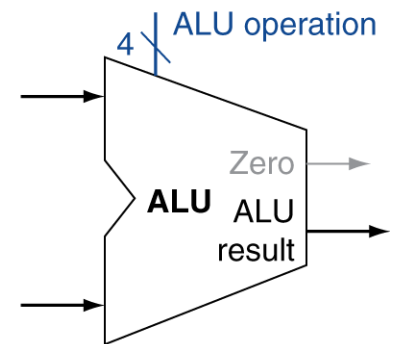


R-type Instructions

- Steps in executing R-type instructions:
 1. Read two register operands
 2. Perform arithmetic/logical operation
 3. Write result in register-file
- I/Os for the register-file



a. Registers



b. ALU